Warehouse Automation

Software Engineering M.Sc. Project

Software Requirements Specification

Authors:

Natai Ella - 208768150
Rodion Novakovskiy - 336121777

Table of Contents

1. Introduction **2**
1.1 Project Overview 2
1.2 Purpose of the System 2
1.3 Scope 2
1.4 Assumptions 2
2. Environment Specification **3**
2.1 Environment Model 3
2.2 Environment Taxonomy 3
2.3 Physical Boundaries 3
2.4 Logical Boundaries 3
2.5 Environmental Constraints 4
3. Agent Specification **4**
3.1 Agent Type 4
3.2 Agent Goals 4
3.3 Perception Capabilities 4
3.4 Action Capabilities 5
3.5 Internal State 5
3.6 Agent Limitations 5
4. Functional Requirements **6**
4.1 System-Level Functional Requirements 6
4.2 Agent-Level Functional Requirements 6
4.3 Coordination Requirements 7
4.4 Learning and Optimization Requirements 7
5. Simulation and Visualization Requirements **8**
5.1 Simulation Requirements 8
5.2 Visualization Requirements 8
5.3 Monitoring and Metrics Visualization 9
5.4 Verification Support 9
6. Interaction Models **9**
6.1 Cooperation Model 9
6.2 Initial Coordination Mechanism 9
6.3 Extended Coordination Mechanism 10
6.4 Conflict Resolution 10
6.5 Optional Role/Zoning Model 10
7. Utility and Success Metrics **11**
7.1 Agent Utility Function 11
7.2 Path Inefficiency 11
7.3 Social Welfare 12
7.4 Global System Success Metrics 12
8. Validation Scenarios **13**

1. Introduction

1.1 Project Overview

This project defines a multi-agent warehouse automation system in which autonomous agents operate in a shared warehouse environment to transport items from pickup locations to designated delivery or storage points. The system is modeled as a grid-based warehouse in which agents must move efficiently, avoid obstacles, coordinate with other agents, and complete logistics tasks under dynamic conditions. The project is intended as a simulation-based study of cooperative multi-agent behavior in a constrained and partially observable environment.

1.2 Purpose of the System

The purpose of the system is to simulate and analyze how multiple cooperative agents can perform warehouse logistics tasks efficiently and safely. The system should support autonomous navigation, task execution, conflict avoidance, and coordination between agents while maintaining high delivery throughput and low congestion.

1.3 Scope

The Software Requirements Specification document (SRS) defines what the warehouse automation system must achieve, including:

- The environment in which agents operate.
- The types of agents and their capabilities.
- The functional requirements of both agents and the overall system.
- The interaction mechanisms used for coordination.
- The simulator and visualization requirements.
- The mathematical utility definitions and success criteria.
- The scenarios used to validate system effectiveness.

1.4 Assumptions

- The warehouse is represented as a finite two-dimensional grid of size X×Y.
- Each agent may perform at most one primary physical action per simulation step: move, pick up, place, or wait. Communication is considered a secondary free action and may occur in addition to the primary action.

- Only one type of mobile warehouse agent is considered.
- Each agent can carry at most one item at a time.

2. Environment Specification

2.1 Environment Model

The warehouse environment shall be represented as a discrete grid of cells. Each cell shall belong to one of the following categories:

- Free traversal cell.
- Pickup location.
- Delivery location.
- A blocked cell such as a shelf or obstacle.

2.2 Environment Taxonomy

The system environment is classified as follows:

- Cooperative - all agents pursue the shared goal of completing delivery tasks.
- Discrete - time progresses in turns or simulation steps.
- Partially observable - each agent can perceive only a limited local neighbourhood.
- Dynamic - the state of the environment may change due to agent movement, task completion, and newly blocked or occupied cells.
- Constrained - no two agents may occupy the same cell at the same time.

2.3 Physical Boundaries

The physical boundaries of the system are the dimensions of the warehouse grid. Agents may not move outside the valid grid area.

2.4 Logical Boundaries

The logical boundaries of the system include:

- The set of active tasks currently available.
- Each agent’s local perception radius,
- Each agent’s internal memory or explored map,

- Rules governing occupancy, movement, carrying, and delivery.
- The warehouse grid size (X × Y) shall be known to all agents at initialization.

2.5 Environmental Constraints

The environment shall enforce the following constraints:

- Blocked cells are not traversable.
- An agent carrying an item may not pick up another item.
- Each item must be associated with a valid destination.
- Simultaneous occupancy of a cell by multiple agents is forbidden.
- Agents may need to replan paths when local conditions change.

3. Agent Specification

3.1 Agent Type

The system shall initially include one agent type: Warehouse Transport Agent.

3.2 Agent Goals

Each warehouse transport agent shall pursue the following individual goal sequence:

1. Receive a transport task.
2. Travel to a valid cell adjacent to the pickup location.
3. Pick up the assigned item from the adjacent pickup cell.
4. Navigate to the item's destination.
5. Place the item at the destination.
6. Become available for the next task.

3.3 Perception Capabilities

Each agent shall be able to perceive:

- Nearby free and blocked cells within its perception radius.
- Nearby agents.
- Nearby pickup and delivery points if visible.

- Whether the intended next cell is occupied.
- The identity and destination of a currently carried item.
- Task information relevant to its assignment.

3.4 Action Capabilities

Each agent shall support the following actions:

- Move one cell in a valid direction.
- Pick up an item from a delivery point.
- Place an item at an adjacent cell.
- Wait for one simulation step.
- Pick up an item from an adjacent pickup cell

3.5 Agent Communication Capabilities

Agents shall support local peer-to-peer communication only with adjacent agents. Two agents may exchange information only when they are located in neighbouring grid cells.

Communication shall be treated as a free action, meaning it does not replace the agent's main action for that simulation step. An agent may still move, pick up, place, or wait during the same turn in which communication occurs.

When two agents become adjacent, they shall share relevant internal knowledge, including:

- explored map information
- known blocked and free cells
- known pickup and delivery locations
- known item locations
- observed agent positions
- known congestion or recently blocked paths
- task-related information when relevant

Each stored piece of map information shall include a timestamp indicating when the information was last observed or updated. When agents exchange information, newer information shall replace older information if both agents have conflicting knowledge about the same cell or object.

This allows agents to gradually improve their local knowledge through interaction while preserving the environment's partially observable nature.

3.6 Internal State

Each agent shall maintain an internal state including:

- Current position.
- Availability status.
- Carrying status.
- Assigned task identifier.
- Partial internal map of explored areas.
- Current route or target cell.

3.7 Agent Limitations

- An agent may perform at most one action per turn.
- An agent may carry only one item at a time.
- An agent has only local perception and does not initially know the full global state.
- An agent must obey collision and occupancy constraints.

3.8 Agent Path Planning and Route Selection

Each agent shall use a non-deterministic, adaptive path-planning strategy rather than always selecting the single shortest path.

The agent shall attempt to move directly toward its current target when the route is clear and valid. However, when the agent detects a blocked cell, obstacle, wall, occupied cell, congestion, or an uncertain unexplored area, it shall evaluate multiple possible route alternatives.

Route selection shall be based on weighted probabilities rather than a fully deterministic rule. For example, if two possible detours are available, the agent may assign a higher probability to the shorter or less congested route, while still maintaining a smaller probability of choosing a longer or less obvious route. This allows agents to explore alternative paths and prevents all agents from always making identical routing decisions.

The pathing mechanism shall consider factors such as:

- estimated distance to the target

- known blocked cells
- Recently observed congestion
- unexplored or uncertain map areas
- previous failed routes
- waiting time caused by other agents
- current task urgency

The weights used for route selection may change dynamically during the simulation. For example, if an agent repeatedly encounters congestion in one area, the probability of selecting that area in the future may decrease. Similarly, if a previously blocked route becomes available, the agent may increase the likelihood of trying it again.

This approach supports partially observable and dynamic warehouse behaviour, since agents do not initially know the full global map and must gradually build internal knowledge while operating. It also aligns with the system requirement that agents may need to replan paths when local conditions change.

4. Functional Requirements

4.1 System-Level Functional Requirements

Functional Requirement	Functional Requirement Description
FR-01	The system shall initialize a warehouse grid with valid cell types, obstacles, pickup points, delivery points, items, and agents
FR-02	The system shall maintain a task list of active pickup-and-delivery jobs
FR-02a	The system shall generate pickup-and-delivery tasks according to a predefined or seed-based deterministic process. Tasks shall be produced as a sequence (queue) of orders, where each order specifies: a pickup location (one of the defined pickup cells), a corresponding delivery location, and an item identifier
FR-02b	The system shall represent each item as an entity with a unique item ID and a spatial location within the warehouse grid
FR-03	The system shall execute the simulation in discrete time steps
FR-04	The system shall prevent illegal moves, including movement into blocked cells, out-of-bounds cells, and currently occupied cells
FR-05	The system shall update the global warehouse state after each simulation step
FR-06	The system shall detect task completion when an assigned item is successfully delivered to its destination
FR-07	The system shall record performance statistics for all agents and for the system as a whole

4.2 Agent-Level Functional Requirements

Functional Requirement	Functional Requirement Description
FR-08	An idle agent shall be able to select or receive an available task
FR-09	An agent shall compute a path toward its current target using available local and stored information

FR-10	An agent shall replan or wait when its next move becomes invalid due to dynamic changes
FR-11	An agent shall pick up an item only when the agent is located in a valid free cell adjacent to the item's pickup cell, is not already carrying another item, and the item is assigned to that agent. The agent shall not occupy the pickup/item cell itself when performing the pickup action.
FR-12	An agent shall place an item only when the agent is located in a valid free cell adjacent to the item's destination cell, and only if the carried item corresponds to that destination. The agent shall not occupy the destination cell itself when performing the placement action.
FR-13	An agent shall update its internal state after movement, pickup, placement, or waiting
FR-14	An agent shall avoid collisions by not entering occupied cells

4.3 Coordination Requirements

Functional Requirement	Functional Requirement Description
FR-15	Multiple agents shall be able to operate concurrently in the same warehouse
FR-16	The system shall support a basic task-allocation policy in which available agents take available tasks
FR-17	The architecture shall support future extension to more advanced allocation methods such as auction-based coordination

4.4 Learning and Optimization Requirements

Functional Requirement	Functional Requirement Description
FR-18	The system shall allow agents to store explored map information
FR-19	The system shall support strategies that prefer shorter or less congested routes

FR-20	The system shall allow future extension to adaptive routing under dynamic obstacles
FR-21	Each agent shall store its perceived and learned environment information as a local internal map, where each stored element (e.g., grid cell state, item location, or observed agent position) is associated with a timestamp indicating the last time the information was observed or updated.
FR-22	When an agent receives new information (through perception or communication), it shall compare timestamps for any overlapping data and retain the most recent information. Older information shall be replaced by newer information when conflicts occur.

5. Simulation and Visualization Requirements

5.1 Simulation Requirements

Requirement	Requirement Description
SVR-01	The system shall provide a simulator that runs the warehouse scenario step by step in real time or controllable simulation time
SVR-02	The simulator shall display the full warehouse grid and all active entities
SVR-03	The simulator shall support starting, pausing, resuming, resetting, and replaying a scenario
SVR-04	The simulator shall support variable simulation speed

5.2 Visualization Requirements

Requirement	Requirement Description
SVR-05	The simulator shall visually distinguish: free cells, blocked cells, pickup points, delivery points, items awaiting pickup, agents, agents carrying items, completed deliveries
SVR-06	The simulator shall show the real-time position of every agent
SVR-07	The simulator shall display each agent's current state, such as: idle, moving to pickup, carrying item, moving to delivery, waiting
SVR-08	The simulator shall display active tasks and their current assignment status
SVR-09	The simulator shall allow an observer to inspect an individual agent and view its: identifier, current task, carried item, local perceived area or known map, path or current intended move

5.3 Monitoring and Metrics Visualization

Requirement	Requirement Description
SVR-10	The simulator shall display live performance indicators including: total completed deliveries, total simulation steps, average task completion time, number of waiting actions, path lengths
SVR-11	The simulator shall log system events such as: task creation, task assignment, pickup event, delivery event, waiting event, route replanning event, blocked path event

5.4 Verification Support

Requirement	Requirement Description
SVR-12	The visualization shall make it possible to verify that agents follow the defined strategies for navigation, task execution, and collision avoidance
SVR-13	The system shall support scenario replay so that an observer can analyze why specific behaviors occurred

6. Interaction Models

6.1 Cooperation Model

The agents operate under a cooperative interaction model. Their objective is collective task completion rather than individual competition. The system shall therefore prioritize overall warehouse efficiency and delivery success over local selfish behavior.

6.2 Initial Coordination Mechanism

Initially, task allocation shall be based on a simple availability mechanism:

- When a task appears, an available agent may take it.
- If multiple agents are available, selection may be based on a simple rule such as first available or nearest visible agent.

6.3 Extended Coordination Mechanism

The system shall be extensible to auction-based task allocation. In this model:

- A new task is announced.
- Eligible agents compute a bid.
- The bid may depend on distance, expected cost, or current availability.
- The task is assigned to the most suitable bidder.

6.4 Conflict Resolution

When multiple agents attempt to move into conflicting regions, the system shall support conflict resolution through one or more of the following:

- Waiting
- Route replanning
- Priority rules
- Zone-based separation
- Reservation of next cells

6.5 Optional Role/Zoning Model

The system may support simple zone identifiers or role partitions to reduce congestion and improve local coordination.

7. Utility and Success Metrics

7.1 Agent Utility Function

Let:

- Ti be the number of tasks completed by agent i
- Si be the total number of steps performed by agent i
- Wi be the number of wait actions by agent i
- Pi be the total path inefficiency of agent i
- Ci be the number of collision violations attributed to agent i

A possible utility function for agent i is:

$$U_i = \alpha T_i - \beta S_i - \gamma W_i - \delta P_i - \epsilon C_i$$

where α, β, γ, δ, ε > 0 are weight constants.

This function rewards successful delivery completion and penalizes wasted motion, waiting, inefficient routes, and unsafe behaviour.

7.2 Path Inefficiency

For each completed delivery k, let:

- $L_{actual}(k)$ be the actual path length,
- $L_{shortest}(S)$ be the shortest feasible path length.

Then the path inefficiency for task k is:

$$PI_k = L_{actual}(k) - L_{shortest}(k)$$

and for agent i:

$$P_i = \sum PI_k \; over\ all\ tasks\ completed\ by\ agent\ i$$

This formalizes the idea of path efficiency by comparing actual travel distance to the shortest feasible path.

7.3 Social Welfare

Let N be the number of agents. The overall social welfare of the system is:

$$SW = \sum_{1}^{N} U_i$$

A higher value of SW indicates better collective performance.

7.4 Global System Success Metrics

The system shall be evaluated using the following metrics:

Completion Time: total simulation steps required to complete all deliveries.

Task Completion Rate (TCR):

TCR = number of completed deliveries / total number of delivery tasks

Agent Efficiency (AEi):

AEi = useful actions of agent i / total actions of agent i

Collision Avoidance Score (CAS):

CAS = 1 - (number of collision violations / total movement attempts)

Throughput (TH):

TH = completed deliveries / simulation time

These metrics are aligned with the success criteria defined for the project and provide a measurable basis for evaluating both individual agent performance and overall system effectiveness.

8. Validation Scenarios

№	Scenario	Description	Purpose
1	Single-Agent Baseline	A single agent operates in a small warehouse with no dynamic obstacles and a fixed set of delivery tasks	Validate the correctness of basic navigation, pickup, and delivery behavior
2	Multi-Agent Cooperative Delivery	Several agents operate simultaneously in the same warehouse with multiple pending tasks	Validate concurrent execution, task distribution, and overall throughput
3	Obstacle - Dense Environment	Agents operate in a warehouse with many blocked cells and narrow corridors	Test pathfinding quality and route replanning under constrained movement
4	Congestion and Conflict Resolution	Multiple agents are directed toward nearby goals that create traffic in shared routes	Test waiting, avoidance, and conflict-resolution behavior
5	Limited Observability	Agents begin with only local knowledge and must gradually build an internal map	Test partially observable decision-making and map-based adaptation
6	Dynamic Change During Execution	A previously free cell becomes blocked or occupied while an agent is en route	Test dynamic replanning and system robustness
7	Allocation Strategy Comparison	The same task set is executed once with simple allocation and once with auction-based allocation	Compare coordination strategies in terms of completion time, throughput, and congestion
8	Scalability Test	The number of agents and tasks is gradually increased	Evaluate how performance changes as system load grows
9	Temporary Agent Trap	An agent becomes temporarily trapped because nearby cells are occupied or blocked by other agents.	Validate that the agent does not permanently give up when all known paths appear blocked. The agent should continue waiting, retrying, updating perception, and replanning when conditions change.

10	Blocked Delivery	A delivery location is surrounded by items, obstacles, or other temporary blockers, preventing direct access.	Validate that the agent can recognize that the target is currently unreachable, continue operating without failure, and retry the task when external changes make access possible again.